



# Language C

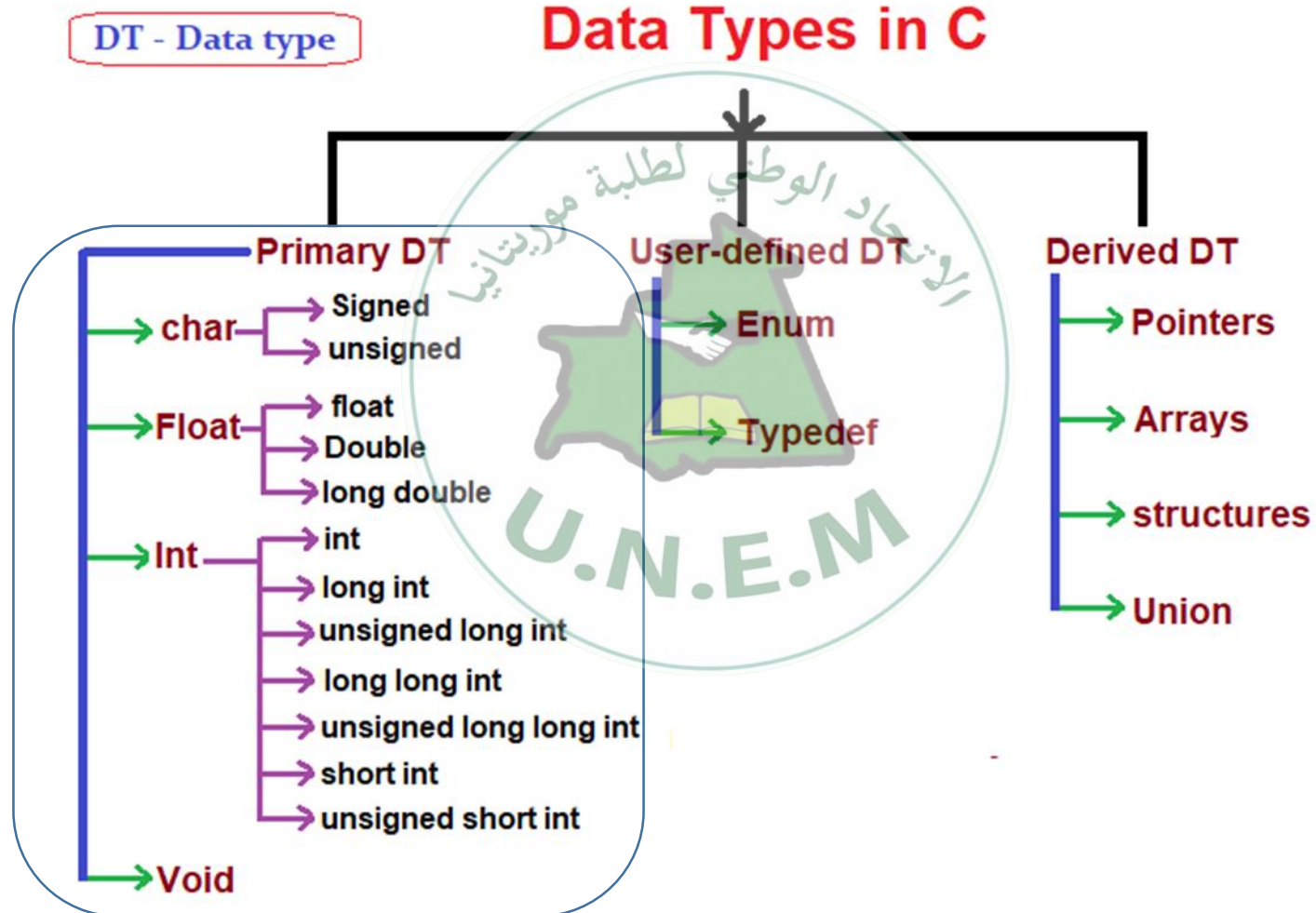
Types de données

Variables

Fonctions Entrée / Sorties



# Data Type





# Primary Data Types

Les types de données primaires sont ceux qui sont déjà définis dans les langages de programmation, également appelés types de données intégrés. Ces types de données sont des blocs de construction de base de tout langage de programmation et de nombreux types de données composites sont construits à partir de ce type de base.

Les types de données primaires incluent :

- Characters
- Integers
- Floating point
- Void



# Primary Data Types

## a) Caractères :

- Un seul caractère en C est une donnée de type « char ». Caractères stockés dans 8 bits ou 1 octet de mémoire et le caractère peut également être signé et non signé. Syntaxe.
- Il n'y a pas de "type de caractère" dédié en langage C. char est identique au type entier. char est juste un plus petit type entier. Ainsi, comme tout autre type entier, il peut être signé ou non signé.



# Primary Data Types

## a) Caractères :

La table ASCII contient 128 valeurs de caractères de 0 à 127 et les 127 caractères restants sont appelés jeu de caractères ASCII étendu.

Dans le jeu étendu de caractères signés, la valeur est comprise entre -128 et -1, tandis que dans le caractère non signé, il prend la valeur de 128 à 255.

Et lorsque vous essayez de donner la valeur 128 au caractère signé, il prendra la valeur -128 comme dans le sens des aiguilles d'une montre.



# Primary Data Types

## b) Integer :

Comme son nom l'indique, utilisé pour contenir uniquement des valeurs entières. Généralement, un entier stocke des valeurs dans une plage limitée de -2147483648 à 2147483647. Un entier signé utilise un bit pour se déconnecter de 16 bits.

Les nombres entiers classés en trois classes de stockage avec des qualificatifs signés et non signés en fonction de la plage de valeurs qu'ils détiennent.



# Primary Data Types

## b) Integer :

| Datatype               | size | Range limit                                 |
|------------------------|------|---------------------------------------------|
| short int              | 2    | -32,768 to 32,767                           |
| unsigned short int     | 2    | 0 to 65,535                                 |
| unsigned int           | 4    | 0 to 4,294,967,295                          |
| int                    | 4    | -2,147,483,648 to 2,147,483,647             |
| long int               | 4    | -2,147,483,648 to 2,147,483,647             |
| unsigned long int      | 4    | 0 to 4,294,967,295                          |
| long long int          | 8    | -(2 <sup>63</sup> ) to (2 <sup>63</sup> )-1 |
| unsigned long long int | 8    | 0 to 18,446,744,073,709,551,615             |



# Primary Data Types

## b) Integer :

```
//Check size of data types
int main(){
printf("long int size = %d\n",sizeof(long int));
printf("short int size = %d\n",sizeof(short int));
printf("unsigned short int size = %d\n",sizeof(unsigned short int));
printf("long int size = %d\n",sizeof(long int));
printf("unsigned long long int size = %d\n",sizeof(unsigned long long int));
return 0;
}
```

```
int main() {
short int a = -32769;
unsigned short int a1 = 65536;
printf("%hd %hd",a,a1);
return 0;
}
```





# Primary Data Types

## c) Float :

Le type de données 'float' utilisé pour stocker les nombres réels en 32 bits avec une précision allant jusqu'à 6 décimales. Le type double également similaire aux magasins flottants en 64 bits avec une précision plus précise jusqu'à 14 décimales et Long double étend encore la précision en utilisant 80 bits et 19 décimales.



# Primary Data Types

c) Float :

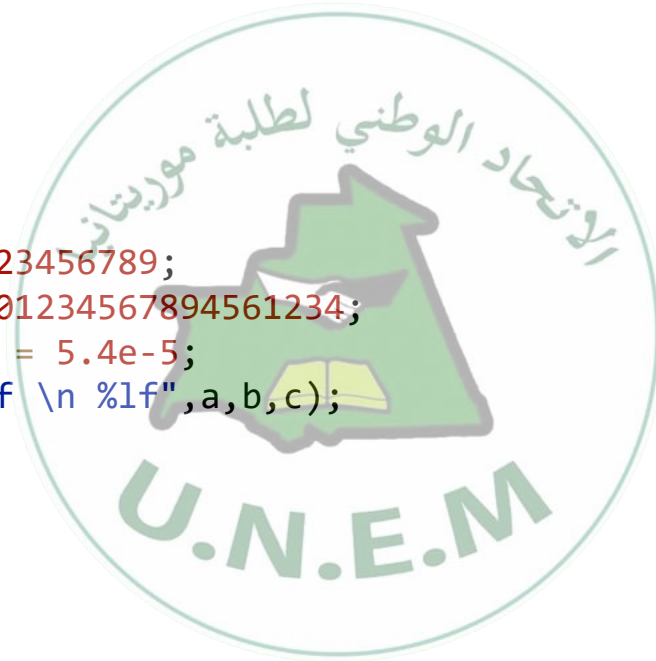
| Datatypes          | Size | Range Limit            |
|--------------------|------|------------------------|
| <b>float</b>       | 4    | 1.2E-38 to 3.4E+38     |
| <b>double</b>      | 8    | 2.3E-308 to 1.7E+308   |
| <b>long double</b> | 12   | 3.4E-4932 to 1.1E+4932 |



# Primary Data Types

c) Float :

```
int main() {  
float a = 1.123456789;  
double b = 1.01234567894561234;  
long double c = 5.4e-5;  
printf("%f %lf \n %lf", a, b, c);  
return 0;  
}
```





# Primary Data Types

d) Void :

- Void est considéré comme un type de données (à des fins d'organisation) et spécifie généralement le type de retour de fonction. Fondamentalement, un mot-clé "void" utilisé comme espace réservé où vous placeriez un type de données pour représenter "NO DATA".
- Void joue un rôle important de programme de type générique pour obtenir un polymorphisme d'exécution un peu similaire aux langages POO.



# Primary Data Types

d) Void :

- Fonction sans valeur de retour et sans argument.

```
void function(void);
```

- Une variable normale ne peut pas être déclarée directement comme void, alors qu'un pointeur peut être void.

```
void fausse_variable; //cette ligne genere une erreur  
void* vague_pointer; // correcte
```



# Variables

- Déclaration d'une variable:

La déclaration d'une variable se fait en utilisant la syntaxe suivante :

`<son type> <son nom>;`

Comme le montre le programme qui suit, il ne faut pas mettre les `<` et `>` comme cela apparaissait dans `<son type> <son nom>;`.

```
#include <stdio.h>

int main () {
    int i; /* déclare un entier de nom i */
    char c; /* déclare un caractère de nom c */
}
```



## 2 Variables

### déclaration d'une variable (Exemple) :

```
#include<stdio.h>
int main () {
int i; /* i : variable de type entier */
int j; /* j : variable de type entier */
i=22; /* i vaut 22 */
j=i; /* on recopie la valeur de i dans j */
/* donc j vaut aussi 22 à présent */
printf ("i vaut %d\n", i); /* Affiche la valeur de i */
printf ("i+j vaut %d\n", i+j); /* Affiche la valeur de i+j */
return 0;
}
```

- `printf ("i vaut %d\n", i);` : %d signifie que l'on attend une valeur entière et qu'il faut l'afficher en décimal (base 10). Le %d sera remplacé par la valeur de i. Cette ligne provoquera donc l'affichage suivant : i vaut 22
- `printf ("i+j vaut %d\n", i+j);` : dans ce cas, %d est remplacé par la valeur de l'expression i+j. Nous obtiendrons l'affichage suivant : i+j vaut 44
- L'exécution complète de ce programme donne donc : i vaut 22 i+j vaut 44



# Variables

- Déclaration d'une variable (Exemple) :

```
#include<stdio.h>
int main () {

char car; /* car: variable de type caractère */
char f; /* f: variable de type caractère */
car='E';
f='e';
printf("car=%c f=%c\n",car,f);

return 0;
}
```





# Fonctions d'Entrees-Sorties

- Il s'agit des fonctions de la librairie standard **stdio.h** utilisées avec les unités classiques d'entrées-sorties, qui sont respectivement le clavier et l'écran.
- Sur certains compilateurs, l'appel à la librairie **stdio.h** par la directive au préprocesseur **#include** n'est pas nécessaire pour utiliser **printf** et **scanf**.



## 3. Fonctions d'Entrees-Sorties

- **Printf**

La fonction **printf** est une fonction d'impression formatée, ce qui signifie que les données sont converties selon le format particulier choisi. Sa syntaxe est :

```
printf("chaîne de contrôle ", expression1, ..., expressionN);
```

La chaîne de contrôle contient le texte à afficher et les spécifications de format correspondant à chaque expression de la liste. Les spécifications de format ont pour but d'annoncer le format des données à visualiser. Elles sont introduites par le caractère **%**, suivi d'un caractère désignant le format d'impression.



# Fonctions d'Entrees-Sorties

- **Printf**

Formats d'impression pour la fonction **printf**

| Format   | Role                                   | Type            |
|----------|----------------------------------------|-----------------|
| %d ou %i | Un entier relatif                      | int             |
| %u       | Un entier naturel (non signé)          | unsigned int    |
| %o       | Un entier sous forme octale            | int             |
| %x       | Un entier sous forme hexadecimal (a-f) | int             |
| %X       | Un entier sous forme hexadecimal (A-F) | int             |
| %f       | Un réel                                | float ou double |
| %e       | Un réel en notation exponentielle (e)  | float ou double |
| %E       | Un reel en notation exponentielle (E)  | float ou double |
| %s       | Une chaine de caracteres               | Char'           |
| %c       | Un seul caractere                      | char            |



# Fonctions d'Entrees-Sorties

- **Scanf**

La fonction **scanf** permet de saisir des données au clavier et de les stocker aux adresses spécifiées par les arguments de la fonctions.

**scanf**("chaîne de contrôle", **argument1**, ..., **argumentN**)

La chaîne de contrôle indique le format dans lequel les données lues sont converties. Elle ne contient pas d'autres caractères (notamment pas de **\n**).

Comme pour **printf**, les conversions de format sont spécifiées par un caractère précédé du signe **%**.



# Fonctions d'Entrees-Sorties

- **Scanf**

```
#include<stdio.h>
```

```
main(void)
```

```
{
```

```
int i;
```

```
printf("entrez un entier sous forme hexadecimale i = ");
```

```
scanf("%x",&i);
```

```
printf("i = %d\n",i);
```

```
}
```

